

docs/project-plan.md: "2026-08-24T18:43:57-03:00"

docs/decisions/technical-decisions-phase-03-videos.md: "2026-08-24T18:54:35-03:00"

docs/phases/phase-02-auth/phase-02-auth.md: "2026-08-24T18:43:57-03:00"

Phase 03 — Upload e Processamento de Vídeos

Objective

Deliver the complete video ingestion and processing backend — direct-to-storage resumable upload of files up to 10GB (presigned multipart), automatic background processing (duration/metadata via ffprobe, thumbnail via ffmpeg) in an isolated worker, unique public URLs, and direct-from-storage streaming and download — establishing the `Video` domain and the

`draft` → `processing` → `ready` | `error` lifecycle that Fase 04+ (management, public pages) will build on. No frontend code is written in this phase; the cross-layer upload (TD-03) and streaming (TD-07) contracts are defined here for future consumption.

Step Implementations

SI-03.1 — Infrastructure: Storage, Queue, and Worker (Docker Compose, Config Namespaces, Dependencies)

Description: Install all Phase 03 production dependencies, add the three new infrastructure containers (`redis` , `minio` , `nestjs-worker`) to Docker Compose, create the `storage` , `queue` , and `upload` config namespaces following the `registerAs` pattern from Fase 01, and extend the Joi validation schema. This SI provides the infrastructure foundation every subsequent Phase 03 SI depends on.

Technical actions:

- Install production dependencies in `nestjs-project`: `@nestjs/bullmq@^11.0.5` , `bullmq@^5.81.3` , `ioredis@^5.11.1` , `@aws-sdk/client-s3@^3` , `@aws-sdk/s3-request-presigner@^3` , `nanoid@^3.3.18` (exact pins and rationale in `library-refs.md` — `nanoid` stays on the CommonJS 3.x line; `bullmq` on the stable 5.x line supported by `@nestjs/bullmq@11`)
- Create `src/config/storage.config.ts` — `registerAs('storage', ...)` reading `STORAGE_ENDPOINT` (string, default `'http://minio:9000'`), `STORAGE_REGION` (string, default `'us-east-1'`), `STORAGE_ACCESS_KEY` (string, required), `STORAGE_SECRET_KEY` (string, required), `STORAGE_BUCKET` (string, default `'videos'`), `STORAGE_FORCE_PATH_STYLE` (boolean, default `true`), `STORAGE_PUBLIC_ENDPOINT` (string, default = `STORAGE_ENDPOINT` — the browser-reachable host baked into presigned URLs)
- Create `src/config/queue.config.ts` — `registerAs('queue', ...)` reading `REDIS_HOST` (string, default `'redis'`), `REDIS_PORT` (number, default `6379`), `VIDEO_QUEUE_NAME` (string, default `'video-processing'`), `VIDEO_JOB_ATTEMPTS` (number, default `3`), `VIDEO_JOB_BACKOFF_MS` (number, default `5000`)
- Create `src/config/upload.config.ts` — `registerAs('upload', ...)` reading `UPLOAD_MAX_SIZE_BYTES` (number, default `10737418240` = 10GB), `UPLOAD_PART_SIZE_BYTES` (number,

default 67108864 = 64MB), `UPLOAD_PRESIGN_TTL_SECONDS` (number, default 3600), `STREAM_PRESIGN_TTL_SECONDS` (number, default 21600), and optional `FFMPEG_PATH` (string, default 'ffmpeg'), `FFPROBE_PATH` (string, default 'ffprobe')

- Update `src/config/env.validation.ts` — add all new environment variables to the Joi schema (`STORAGE_ACCESS_KEY` / `STORAGE_SECRET_KEY` required, others with defaults matching the Compose service names). Update `.env.example` with all new variables and Docker Compose-compatible defaults
- Add `redis` service to `nestjs-project/compose.yaml` — image `redis:7-alpine` , port 6379 , `command`: ["redis-server", "--appendonly", "yes"] (AOF persistence), a volume for `/data` , healthcheck `redis-cli ping`
- Add `minio` service — image `minio/minio` , `command`: `server /data --console-address ":9001"` , ports 9000 (API) and 9001 (console), env `MINIO_ROOT_USER` / `MINIO_ROOT_PASSWORD` , a volume for `/data` , healthcheck on `/minio/health/live` . Add a one-shot `minio-setup` service (image `minio/mc`) that waits for MinIO and creates the `videos` bucket idempotently
- Add `nestjs-worker` service — built from the same `Dockerfile.dev` image and codebase (shared volume), with a distinct `command` running the worker entrypoint (SI-03.4), `depends_on` `redis` (healthy), `minio` (healthy), and `db` (healthy); no published HTTP port. Ensure the worker image has `ffmpeg` and `ffprobe` available (installed via the Dockerfile or an `apt` step in the worker command for the dev image)
- Update `nestjs-api` `depends_on` to include `redis` and `minio` (condition `service_healthy`)

Dependencies: None

Acceptance criteria:

- Application starts without errors when all new environment variables are provided — existing E2E test (`GET /` returns 200) still passes
- Starting the application without `STORAGE_ACCESS_KEY` or `STORAGE_SECRET_KEY` causes a Joi validation error at bootstrap — the app does not start
- `docker compose up` brings up `redis` , `minio` , `minio-setup` , `nestjs-api` , and `nestjs-worker` ; the `videos` bucket exists in MinIO (visible in the console at `localhost:9001`) after `minio-setup` completes
- Redis is reachable from the API and worker containers at `redis:6379` ; MinIO is reachable at `minio:9000` — both addressed by service name, never `localhost`
- `ffprobe -version` and `ffmpeg -version` succeed inside the `nestjs-worker` container

SI-03.2 — Video Entity, Migration, and VideosModule

Description: Create the `Video` entity with the `draft` → `processing` → `ready` | `error` status enum (TD-08), an internal `uuid` primary key, a unique nanoid `public_id` (TD-06), storage-key columns, and extracted-metadata columns. Establish the `ManyToOne` relation to `Channel` . Generate the migration and wire the `VideosModule` .

Technical actions:

- Create `src/videos/entities/video.entity.ts` — `@Entity('videos')` with columns: `id` (uuid PK generated), `public_id` (varchar, unique — nanoid, TD-06), `title` (varchar(255)), `description` (text, nullable), `status` (enum `VideoStatus` , default 'draft' — TD-08), `channel_id` (uuid FK → channels, not null), `storage_key` (varchar — object key of the source video), `thumbnail_key` (varchar, nullable — object key of the generated thumbnail), `duration_seconds` (int, nullable —

from `ffprobe`), `metadata` (jsonb, nullable — raw `ffprobe` format / streams summary), `file_size_bytes` (bigint, nullable), `mime_type` (varchar, nullable), `original_filename` (varchar, nullable), `upload_id` (varchar, nullable — S3 multipart `UploadId` while in draft), `created_at` (CreateDateColumn), `updated_at` (UpdateDateColumn). Define `@ManyToOne()` => `Channel` with `@JoinColumn({ name: 'channel_id' })` and add an index on `channel_id`

- Create `src/videos/entities/video-status.enum.ts` — export `enum VideoStatus { DRAFT = 'draft', PROCESSING = 'processing', READY = 'ready', ERROR = 'error' }`
- Create `src/videos/public-id.util.ts` — export `generatePublicId(): string` using `nanoid(12)` (12 chars, ~71 bits entropy — safe against collision at platform scale; unique index is the backstop). Import `nanoid` from the CommonJS 3.x build
- Generate migration via `npm run migration:generate -- src/database/migrations/CreateVideos` and review the generated SQL for the enum type, columns, `public_id` unique index, `channel_id` index and FK, and `bigint / jsonb` column types
- Create `src/videos/videos.module.ts` — `VideosModule` with `TypeOrmModule.forFeature([Video])` in imports, importing `ChannelsModule` (for ownership resolution) and exporting `TypeOrmModule` so the worker can access the repository

Tests:

File	Layer	Verifies
<code>src/videos/entities/video.entity.integration-spec.ts</code>	Integration	Unique <code>public_id</code> constraint; <code>status</code> defaults to <code>'draft'</code> ; enum rejects invalid values; <code>channel_id</code> FK relation; <code>metadata</code> jsonb round-trips; <code>file_size_bytes</code> stores $> 2^{31}$ (bigint); nullable columns accept null; timestamps auto-populated
<code>src/videos/public-id.util.spec.ts</code>	Unit	<code>generatePublicId</code> returns a 12-char URL-safe id; two calls produce different ids
<code>src/videos/videos.module.spec.ts</code>	Unit	Module compiles with <code>TypeOrmModule.forFeature([Video])</code> and <code>ChannelsModule</code> wiring

Dependencies: SI-03.1

Acceptance criteria:

- `npm run migration:run` creates the `videos` table with all columns, the `video_status` enum type, the unique `public_id` index, and the `channel_id` index + FK
- Inserting a video with a duplicate `public_id` fails with a unique constraint violation
- A newly created video has `status = 'draft'` by default
- `status` accepts only `'draft'`, `'processing'`, `'ready'`, `'error'` — any other value is rejected by the enum constraint

- `file_size_bytes` correctly stores a value larger than 2 GB (bigint, not int)
- A video row references an existing channel via `channel_id`; deleting is constrained by the FK

SI-03.3 — Upload Initiation: Draft Pre-Registration and Presigned Multipart (TD-03)

Description: Implement the presigned multipart upload handshake (TD-03): `POST /videos` pre-registers a `draft` and calls `CreateMultipartUpload`; `POST /videos/:publicId/parts/:partNumber/url` returns a presigned `UploadPartCommand` URL; `POST /videos/:publicId/complete` calls `CompleteMultipartUpload` and transitions the video to `processing` (job publish happens in SI-03.4). File bytes never traverse the API. All endpoints require authentication and enforce channel ownership.

Technical actions:

- Create `src/storage/storage.module.ts` and `src/storage/storage.service.ts` — `StorageService` injecting `storageConfig`, constructing a single `S3Client` (`region`, `endpoint`, `credentials`, `forcePathStyle: true` per TD-02). Implement: `createMultipartUpload(key, contentType): Promise<{ uploadId }>`; `getPresignedUploadPartUrl(key, uploadId, partNumber, ttl): Promise<string>` (presign `UploadPartCommand` via `@aws-sdk/s3-request-presigner`); `completeMultipartUpload(key, uploadId, parts): Promise<void>` (`parts = { ETag, PartNumber }[]`); `abortMultipartUpload(key, uploadId): Promise<void>`; `getPresignedGetUrl(key, ttl, { disposition? }): Promise<string>` (used by SI-03.5); `deleteObject(key): Promise<void>` (used by SI-03.6). Export `StorageService`
- Create `src/videos/videos.service.ts` — `VideosService` injecting `Repository<Video>`, `StorageService`, `ChannelsService`, and `uploadConfig`. Implement `initiateUpload(userId, dto): Promise<{ publicId, uploadId, storageKey, partSize }>` — resolve the caller's channel via `ChannelsService` (throw `NotVideoOwnerException` if the target `channelId` is not owned by `userId`), validate `fileSize <= UPLOAD_MAX_SIZE_BYTES` (throw `FileTooLargeException`), generate `public_id` (retry on unique-violation like the nickname pattern in `ChannelsService`), derive `storage_key = channelId + '/' + publicId + '/source'`, call `storageService.createMultipartUpload`, persist the `draft` row with `upload_id`, `file_size_bytes`, `mime_type`, `original_filename`, `title`, return identifiers + configured `partSize`
- Implement `getPartUploadUrl(userId, publicId, partNumber): Promise<{ url, expiresIn }>` — load video by `public_id`, assert owner + `status = 'draft' + upload_id` present (else `UploadNotInitiatedException`), validate `partNumber` in `1..10000`, return presigned URL with `UPLOAD_PRESIGN_TTL_SECONDS`
- Implement `completeUpload(userId, publicId, dto): Promise<VideoResponse>` — load + assert owner + `status = 'draft'` (else `InvalidStatusTransitionException`; if already `processing / ready` throw `UploadAlreadyCompletedException`), call `storageService.completeMultipartUpload` with the client-supplied sorted `parts`, set `status = 'processing'`, clear `upload_id`, save. (Job enqueue added in SI-03.4.) Return the video projection
- Create DTOs: `src/videos/dto/initiate-upload.dto.ts` (`title @IsString()@MaxLength(255)`, optional `description`, `channelId @IsUUID()`, `fileSize @IsInt()@IsPositive()`, `contentType @IsString()`, optional `originalFilename`); `src/videos/dto/complete-upload.dto.ts` (`parts: @IsArray()@ValidateNested({each:true}) of { partNumber @IsInt(), eTag @IsString() }`)
- Create `src/videos/videos.controller.ts` — route prefix `'videos'`, protected by the inherited global `JwtAuthGuard` (no `@Public()`). `@Post()` → `initiateUpload` (201); `@Post('/:publicId/parts/:partNumber/url')` → `getPartUploadUrl` (200); `@Post('/:publicId/complete')` → `completeUpload` (200). Owner id read via `@CurrentUser()`

- Add video-domain exceptions to `src/common/exceptions/` extending `DomainException`: `VideoNotFoundException` (404), `NotVideoOwnerException` (403), `UploadNotInitiatedException` (409), `UploadAlreadyCompletedException` (409), `InvalidStatusTransitionException` (409), `FileTooLargeException` (413)

Tests:

File	Layer	Verifies
<code>src/storage/storage.service.integration-spec.ts</code>	Integration	Against MinIO: <code>createMultipartUpload</code> returns an <code>uploadId</code> ; a presigned part URL accepts a <code>PUT</code> ; <code>completeMultipartUpload</code> assembles the object; <code>abortMultipartUpload</code> removes it
<code>src/videos/videos.service.spec.ts</code>	Unit	<code>initiateUpload</code> rejects non-owned channel (403) and oversized file (413), generates <code>public_id</code> , persists draft; <code>getPartUploadUrl</code> rejects wrong status; <code>completeUpload</code> transitions draft→processing and rejects double-complete
<code>src/videos/videos.service.integration-spec.ts</code>	Integration	Draft row persisted with <code>upload_id</code> and <code>status='draft'</code> ; <code>completeUpload</code> sets <code>status='processing'</code> and clears <code>upload_id</code> in DB
<code>test/videos.e2e-spec.ts</code>	E2E	<code>POST /videos</code> 401 without token, 201 with token returns <code>{ publicId, uploadId, storageKey, partSize }</code> ; part-url endpoint 200; <code>complete</code> 200 → status <code>processing</code> ; non-owner 403; oversized 413

Dependencies: SI-03.2

Acceptance criteria:

- `POST /videos` with a valid token and an owned `channelId` returns 201 with `{ publicId, uploadId, storageKey, partSize }`; a `draft` video row is persisted with the multipart `upload_id`
- `POST /videos` for a `channelId` not owned by the caller returns 403 `NOT_VIDEO_OWNER`

- `POST /videos` with `fileSize` above 10GB returns 413 `FILE_TOO_LARGE`
- `POST /videos/:publicId/parts/:partNumber/url` returns a presigned URL that accepts a direct `PUT` of a part to MinIO (bytes do not pass through the API)
- `POST /videos/:publicId/complete` with the collected part `ETag`s assembles the object in MinIO and transitions the video to `processing`
- Calling `complete` twice returns 409 `UPLOAD_ALREADY_COMPLETED`
- All upload endpoints return 401 without a valid access token

SI-03.4 — Background Processing: BullMQ Queue, Worker Entrypoint, `ffprobe` + `ffmpeg` (TD-01/04/05/08)

Description: Wire the BullMQ `video-processing` queue (TD-01), publish a `process-video` job when an upload completes, and implement the isolated worker (TD-04) that consumes the job: it downloads/streams the source, runs `ffprobe` for duration/metadata and `ffmpeg` for a single-frame thumbnail via direct `child_process` (TD-05), uploads the thumbnail, and drives the status lifecycle `processing` → `ready` | `error` (TD-08) with BullMQ retries (3 attempts, exponential backoff) and the failed set as dead-letter.

Technical actions:

- Register the queue: in `VideosModule`, add `BullModule.forRootAsync` (inject `queueConfig` → `connection: { host: REDIS_HOST, port: REDIS_PORT }`) and `BullModule.registerQueue` (name: `VIDEO_QUEUE_NAME`, `defaultJobOptions: { attempts: VIDEO_JOB_ATTEMPTS, backoff: { type: 'exponential', delay: VIDEO_JOB_BACKOFF_MS }, removeOnComplete: true, removeOnFail: false }`) (keeping failed jobs = dead-letter set)
- Producer: inject `@InjectQueue(VIDEO_QUEUE_NAME)` `private queue: Queue` into `VideosService`; in `completeUpload`, after setting `status = 'processing'`, enqueue `queue.add('process-video', { publicId, storageKey } satisfies ProcessVideoJob)`. Define `src/videos/jobs/process-video.job.ts` — `interface ProcessVideoJob { publicId: string; storageKey: string }`
- Worker processor: create `src/videos/processors/video-processing.processor.ts` — `@Processor(VIDEO_QUEUE_NAME)` class `VideoProcessingProcessor` extends `WorkerHost` implementing `process(job: Job<ProcessVideoJob>): (1) load video by public_id; (2) obtain a local readable source (download the object to a temp path via StorageService, or a presigned GET consumed by ffmpeg/ffprobe); (3) run ffprobe → parse duration_seconds + metadata; (4) run ffmpeg → extract one frame to a temp jpg; (5) upload the thumbnail to thumbnail_key = channelId + '/' + publicId + '/thumb.jpg'; (6) set duration_seconds, metadata, thumbnail_key, status = 'ready', save; (7) clean temp files. Implement onFailed (via @OnWorkerEvent('failed')) that, only when job.attemptsMade >= job.opts.attempts, sets the video status = 'error' (final failure ⇒ dead-letter)`
- `ffprobe/ffmpeg` command builders: create `src/videos/processors/ffmpeg.util.ts` — `probeVideo(inputPath): Promise<{ durationSeconds, metadata }>` running `execFile(FFPROBE_PATH, ['-v', 'quiet', '-print_format', 'json', '-show_format', '-show_streams', inputPath])` and JSON-parsing `stdout`; `extractThumbnail(inputPath, outputPath, atSeconds): Promise<void>` running `execFile(FFMPEG_PATH, ['-ss', String(atSeconds), '-i', inputPath, '-frames:v', '1', '-q:v', '2', '-y', outputPath])`. Use `util.promisify(execFile)`; surface non-zero exit codes + `stderr` as errors
- Worker entrypoint: create `src/worker.ts` — bootstrap a Nest application context (`NestFactory.createApplicationContext(WorkerModule)`) that imports `config`, `TypeORM`, `StorageModule`, and

`VideosModule` (registering the processor) but starts no HTTP server. Create `src/worker.module.ts` composing only the modules the processor needs. Add npm scripts `start:worker` (`node dist/worker`) and `start:worker:dev` (`ts-node-dev / nest start` variant). The `nestjs-worker` Compose service (SI-03.1) runs this endpoint

- Ensure the worker image provides `ffmpeg / ffprobe` (installed in the Dockerfile/worker command); `FFMPEG_PATH / FFPROBE_PATH` from `uploadConfig` allow overriding the binary locations

Tests:

File	Layer	Verifies
<code>src/videos/processors/ffmpeg.util.spec.ts</code>	Unit	Command builders call <code>exec-File</code> with the exact <code>ffprobe/ffmpeg</code> arguments (mocked <code>child_process</code>); <code>ffprobe</code> JSON is parsed into <code>duration-Seconds + metadata</code> ; non-zero exit rejects
<code>src/videos/processors/video-processing.processor.spec.ts</code>	Unit	On success sets <code>status='ready'</code> with <code>duration/thumbnail</code> ; onFailed sets <code>status='error'</code> only after the final attempt; earlier attempts leave <code>status='processing'</code>
<code>src/videos/videos.service.spec.ts</code> (extended)	Unit	<code>completeUpload</code> enqueues a <code>process-video</code> job with <code>{ publicId, storageKey }</code>
<code>src/videos/video-processing.integration.spec.ts</code>	Integration	Producer enqueues → processor consumes against real Redis + MinIO: a seeded object is probed, a thumbnail is written to MinIO, and the DB row reaches <code>status='ready'</code>

Dependencies: SI-03.3

Acceptance criteria:

- Completing an upload enqueues a `process-video` job on the `video-processing` queue
- The `nestjs-worker` container consumes the job, extracts `duration + metadata` (`ffprobe`), generates a thumbnail (`ffmpeg`), uploads the thumbnail to storage, and sets the video `status = 'ready'` with `duration_seconds`, `metadata`, and `thumbnail_key` populated
- A job that throws is retried up to 3 times with exponential backoff; the video stays `processing` across retries

- After the final failed attempt the video `status` becomes `error` and the job remains in the BullMQ failed set (dead-letter)
- FFmpeg CPU work runs in the worker process only — the API container does not invoke `ffmpeg/ffprobe`

SI-03.5 — Streaming and Download (Presigned GET, native Range/206) (TD-07)

Description: Implement direct-from-storage streaming and download (TD-07): `GET /videos/:publicId/stream` issues a short-lived presigned GET URL (MinIO/S3 natively serves `Range / 206`) and `302`-redirects the client to it; `GET /videos/:publicId/download` issues a presigned URL with a `Content-Disposition: attachment` override. Streaming is gated on `status = 'ready'`.

Technical actions:

- Implement `getStreamUrl(publicId): Promise<string>` in `VideosService` — load video by `public_id` (throw `VideoNotFoundException` if absent), assert `status = 'ready'` (else `VideoNotReadyException`), return `storageService.getPresignedGetUrl(storage_key, STREAM_PRESIGN_TTL_SECONDS)` built against `STORAGE_PUBLIC_ENDPOINT` so the URL is browser-reachable
- Implement `getDownloadUrl(publicId): Promise<string>` in `VideosService` — same gating, but pass a `ResponseContentDisposition: 'attachment; filename="<original_or_title>.mp4"'` override to the presigner
- Add to `VideosController`: `@Public() @Get('/:publicId/stream')` with `@Redirect()` (or manual `res.redirect(302, url)`) returning the presigned stream URL; `@Public() @Get('/:publicId/download')` with `@Redirect()` returning the presigned attachment URL. (Public read access for `ready` videos this phase; per-viewer auth can be layered later per TD-07.)
- Confirm MinIO serves `Range` requests through the presigned URL (returns `206 Partial Content + Accept-Ranges: bytes`) — asserted in the E2E/integration test

Tests:

File	Layer	Verifies
<code>src/videos/videos.service.spec.ts</code> (extended)	Unit	<code>getStreamUrl / getDownloadUrl</code> throw <code>VideoNotReadyException</code> for non- ready status and <code>VideoNotFoundException</code> for unknown id; download passes an attachment disposition
<code>src/videos/streaming.integration.spec.ts</code>	Integration	Against MinIO with a seeded ready object: the presigned stream URL honors a <code>Range: bytes=0-1023</code> request with <code>206 + Accept-Ranges: bytes</code> ; the download URL responds with <code>Content-Disposition: attachment</code>
<code>test/videos.e2e.spec.ts</code> (extended)	E2E	<code>GET /videos/:publicId/stream</code> returns 302 to a presigned URL for a ready video; returns 409 <code>VIDEO_NOT_READY</code> for a processing / draft video; 404 for an unknown id; <code>GET /videos/:publicId/download</code> returns 302 to an attachment URL

Dependencies: SI-03.4

Acceptance criteria:

- `GET /videos/:publicId/stream` for a ready video returns a 302 redirect to a short-lived presigned GET URL served directly by MinIO/S3
- The presigned stream URL honors HTTP `Range` requests, returning `206 Partial Content` with `Accept-Ranges: bytes` — the browser can seek without downloading the whole file
- `GET /videos/:publicId/stream` for a video that is not ready returns 409 `VIDEO_NOT_READY`
- `GET /videos/:publicId/download` returns a 302 redirect to a presigned URL that forces `Content-Disposition: attachment`
- `GET /videos/:publicId/stream` for an unknown `publicId` returns 404 `VIDEO_NOT_FOUND`
- Video bytes are served by storage directly — no video payload flows through the API

SI-03.6 — Complementary REST: Retrieve, List, Update, Delete

Description: Complete the `Video` REST surface: public retrieval of a `ready` video's metadata, listing a channel's videos, owner-only metadata update (title/description), and owner-only deletion with storage cleanup (delete source + thumbnail, and abort any in-flight multipart upload).

Technical actions:

- Implement `findByPublicId(publicId): Promise<VideoResponse>` in `VideosService` — load by `public_id`, throw `VideoNotFoundException` if absent, assert `status = 'ready'` for public access (else `VideoNotReadyException`); return a projection (no `upload_id` / `storage_key` leaked — expose `publicId`, `title`, `description`, `status`, `durationSeconds`, `thumbnailUrl` (presigned), `channelId`, `timestamps`)
- Implement `listByChannel(channelId): Promise<VideoResponse[]>` — return that channel's videos ordered by `created_at` DESC (public listing returns only `ready`; the owner sees all statuses when authenticated — resolved via the optional authenticated user)
- Implement `update(userId, publicId, dto): Promise<VideoResponse>` — load + assert owner (`NotVideoOwnerException`), update `title` / `description` only, save, return projection
- Implement `remove(userId, publicId): Promise<void>` — load + assert owner; if `status = 'draft'` and `upload_id` present, `storageService.abortMultipartUpload`; delete `storage_key` and `thumbnail_key` objects via `storageService.deleteObject`; delete the DB row
- Create `src/videos/dto/update-video.dto.ts` — `UpdateVideoDto` with optional `title @IsString()@MaxLength(255)` and optional `description @IsString()`
- Add to `VideosController`: `@Public() @Get('/:publicId') → findByPublicId (200)`; `@Public() @Get('/channels/:channelId')` (or a `GET /channels/:channelId/videos` route) `→ listByChannel (200)`; `@Patch('/:publicId') → update (200, authenticated + owner)`; `@Delete('/:publicId') → remove (204, authenticated + owner)`

Tests:

File	Layer	Verifies
src/videos/ videos.service.spec.ts (extended)	Unit	update / remove reject non-owner (403); remove aborts multipart when draft and deletes both storage keys; findByPublicId hides internal fields and gates on ready
src/videos/ videos.service.integration-spec.ts (extended)	Integration	update persists new title/description; remove deletes the row and calls storage cleanup; listByChannel orders by created_at DESC
test/videos.e2e-spec.ts (extended)	E2E	GET /videos/:publicId 200 for ready / 409 for not-ready / 404 unknown; GET /channels/:channelId/videos 200 list; PATCH 200 owner / 403 non-owner / 401 no token; DELETE 204 owner / 403 non-owner

Dependencies: SI-03.5

Acceptance criteria:

- GET /videos/:publicId returns 200 with the public video projection for a ready video; 409 VIDEO_NOT_READY otherwise; 404 VIDEO_NOT_FOUND for an unknown id — internal fields (storage_key , upload_id) are never exposed
- GET /channels/:channelId/videos returns the channel's videos ordered newest-first
- PATCH /videos/:publicId by the owner updates title / description and returns 200; a non-owner gets 403 NOT_VIDEO_OWNER ; no token gets 401
- DELETE /videos/:publicId by the owner returns 204, removes the DB row, deletes the source and thumbnail objects from storage, and aborts any in-flight multipart upload; a non-owner gets 403
- Deleting a draft video that never completed its upload aborts the multipart upload (no orphaned parts left in storage)

Technical Specifications

Data Model

Video

Column	Type	Constraints	Notes
id	uuid	PK, generated	Internal primary key (TD-06)
public_id	varchar	unique, not null	nanoid(12), URL-safe public identifier (TD-06)
title	varchar(255)	not null	Provided on upload initiation
description	text	nullable	Optional
status	enum <code>video_status</code>	not null, default <code>'draft'</code>	<code>draft</code> / <code>processing</code> / <code>ready</code> / <code>error</code> (TD-08)
channel_id	uuid	FK → channels.id, not null	Owning channel
storage_key	varchar	not null	Object key of the source video in storage
thumbnail_key	varchar	nullable	Object key of the generated thumbnail (set by worker)
duration_seconds	int	nullable	Extracted by ffprobe (set by worker)
metadata	jsonb	nullable	Summarized ffprobe format / streams output
file_size_bytes	bigint	nullable	Declared source size (bigint — supports > 2GB / up to 10GB)
mime_type	varchar	nullable	Declared content type
original_filename	varchar	nullable	Client-provided filename
upload_id	varchar	nullable	

Column	Type	Constraints	Notes
			S3 multipart Upload-Id ; set while draft , cleared on complete
created_at	timestamp	not null, auto-generated	@CreateDateColumn
updated_at	timestamp	not null, auto-generated	@UpdateDateColumn

Relations: Video → Channel (many-to-one via `channel_id`)

Indexes: (`public_id`) — unique, (`channel_id`) — FK/listing

Status transitions (TD-08): `draft` → `processing` (on `complete-upload` + job publish); `processing` → `ready` (job success); `processing` → `error` (job permanently failed after final retry); `error` → `processing` (on re-submit — supported by the enum, no dedicated endpoint this phase).

API Contracts

All `/videos` mutating endpoints are protected by the inherited global `JwtAuthGuard` ; read endpoints (`stream` , `download` , `GET /videos/:publicId` , channel listing) are `@Public()` . Ownership is enforced by resolving the caller's channel via `ChannelsService` .

POST /videos (SI-03.3)

Request headers:

- Authorization: Bearer
- Content-Type: application/json

Request body:

- title: string, required — max 255
- description: string, optional
- channelId: string (uuid), required — must be owned by the caller
- fileSize: integer, required — bytes, must be ≤ 10737418240 (10GB)
- mimeType: string, required
- originalFilename: string, optional

Response 201:

- publicId: string (nanoid)
- uploadId: string (S3 multipart UploadId)
- storageKey: string
- partSize: integer (bytes — client chunk size)

Error responses:

- 401: missing/invalid access token
- 403 NOT_VIDEO_OWNER: `channelId` not owned by the caller
- 413 FILE_TOO_LARGE: `fileSize` exceeds the 10GB limit
- 400 validation error: body fails schema validation

POST /videos/:publicId/parts/:partNumber/url (SI-03.3)**Request headers:**

- Authorization: Bearer

Path parameters:

- publicId: string, required
- partNumber: integer, required — 1..10000

Response 200:

- url: string (presigned UploadPart URL — client PUTs the chunk directly to storage)
- expiresIn: integer (seconds)

Error responses:

- 401: missing/invalid access token
 - 403 NOT_VIDEO_OWNER: caller does not own the video
 - 404 VIDEO_NOT_FOUND: unknown publicId
 - 409 UPLOAD_NOT_INITIATED: video is not in draft / has no active multipart upload
-

POST /videos/:publicId/complete (SI-03.3)**Request headers:**

- Authorization: Bearer
- Content-Type: application/json

Request body:

- parts: array, required — objects { partNumber: integer, eTag: string }

Response 200:

- publicId, title, description, status (processing), channelId, createdAt, updatedAt

Error responses:

- 401 / 403 NOT_VIDEO_OWNER / 404 VIDEO_NOT_FOUND
 - 409 UPLOAD_ALREADY_COMPLETED: video already processing / ready
 - 409 INVALID_STATUS_TRANSITION: video not in draft
 - 400 validation error
-

GET /videos/:publicId/stream (SI-03.5)

Path parameters: publicId: string, required

Response 302: Redirect (Location) to a short-lived presigned GET URL served directly by MinIO/S3 (native Range / 206).

Error responses:

- 404 VIDEO_NOT_FOUND: unknown publicId
 - 409 VIDEO_NOT_READY: video status is not ready
-

GET /videos/:publicId/download (SI-03.5)

Path parameters: publicId: string, required

Response 302: Redirect to a presigned GET URL with `response-content-disposition: attachment`.

Error responses:

- 404 VIDEO_NOT_FOUND
 - 409 VIDEO_NOT_READY
-

GET /videos/:publicId (SI-03.6)

Path parameters: publicId: string, required

Response 200:

- publicId, title, description, status, durationSeconds, thumbnailUrl (presigned), channelId, createdAt, updatedAt

Error responses:

- 404 VIDEO_NOT_FOUND
 - 409 VIDEO_NOT_READY: for public access to a non-`ready` video
-

GET /channels/:channelId/videos (SI-03.6)

Path parameters: channelId: string (uuid), required

Response 200:

- array of video projections, ordered by `createdAt` descending (public callers see only `ready` videos)

Error responses:

- 404: unknown channel (optional — empty list acceptable)
-

PATCH /videos/:publicId (SI-03.6)

Request headers:

- Authorization: Bearer
- Content-Type: application/json

Request body:

- title: string, optional — max 255
- description: string, optional

Response 200: updated video projection.

Error responses:

- 401 / 403 NOT_VIDEO_OWNER / 404 VIDEO_NOT_FOUND / 400 validation error
-

DELETE /videos/:publicId (SI-03.6)

Request headers:

- Authorization: Bearer

Response 204: No content — DB row deleted, source + thumbnail objects removed, any in-flight multipart upload aborted.

Error responses:

- 401 / 403 NOT_VIDEO_OWNER / 404 VIDEO_NOT_FOUND

Validation Rules — Upload and Update

Field	Rule	Error message
title	Non-empty, max 255 chars	title must be shorter than or equal to 255 characters
channelId	Must be a valid UUID	channelId must be a UUID
fileSize	Positive integer $\leq$ 10737418240	fileSize must not be greater than 10737418240
partNumber	Integer 1..10000	partNumber must not be greater than 10000
parts	Non-empty array of { part-Number, eTag }	parts should not be empty

Authorization Matrix

Endpoint	Public	Authenticated	Owner-only	Notes
POST /videos		✓	✓	Must own the target <code>channelId</code>
POST /videos/:publicId/parts/:partNumber/url		✓	✓	
POST /videos/:publicId/complete		✓	✓	
GET /videos/:publicId/stream	✓			Only <code>ready</code> videos
GET /videos/:publicId/download	✓			Only <code>ready</code> videos
GET /videos/:publicId	✓			Only <code>ready</code> videos publicly
GET /channels/:channelId/videos	✓			Public listing returns <code>ready</code> only
PATCH /videos/:publicId		✓	✓	
DELETE /videos/:publicId		✓	✓	

Error Catalog

Error response format: (inherited from Fase 02 — applies to all nestjs-project HTTP endpoints)

```
{ statusCode: number, error: string, message: string }
```

The `error` field carries the domain error code from the catalog (e.g., `"VIDEO_NOT_FOUND"`). For validation errors, `error` is `"VALIDATION_ERROR"` and `message` is an array of field-level error strings. All video-domain exceptions extend the inherited `DomainException` base and are rendered by the global `DomainExceptionFilter`.

Code	HTTP	Message	Trigger
VIDEO_NOT_FOUND	404	Video not found	Any endpoint with a <code>publicId</code> not present in the videos table
NOT_VIDEO_OWNER	403	You do not own this resource	Mutating a video/channel not owned by the authenticated caller
FILE_TOO_LARGE	413	File exceeds the maximum allowed size	POST /videos with <code>fileSize</code> > 10GB
UP-LOAD_NOT_INITIATED	409	Upload has not been initiated for this video	Requesting a part URL when the video is not a <code>draft</code> with an active multipart upload
UP-LOAD_ALREADY_COMPLETED	409	Upload has already been completed	POST /videos/:publicId/complete on a video already <code>processing</code> / <code>ready</code>
INVALID_STATUS_TRANSITION	409	Invalid status transition	An operation not allowed from the video's current status
VIDEO_NOT_READY	409	Video is not ready	Streaming/downloading/public-fetching a video whose status is not <code>ready</code>

Events/Messages

process-video

Payload:

```
{ "publicId": "string", "storageKey": "string" }
```

Producer: `VideosService` (API container) — enqueues on the `video-processing` queue after `CompleteMultipartUpload` (per `phase-03-videos/TD-01`, `phase-03-videos/TD-03`)

Consumer: `VideoProcessingProcessor` (`nestjs-worker` container) — `@Processor('video-pro-`

cessing') extending WorkerHost (per phase-03-videos/TD-04 , phase-03-videos/TD-05)

Trigger: Fires when an upload is completed (POST /videos/:publicId/complete transitions the video draft → processing)

Delivery semantics: at-least-once — BullMQ retries up to 3 attempts with exponential backoff; the job handler is idempotent (re-probing/re-thumbnailing overwrites the same storage keys and DB fields). After the final failed attempt the video is set to error and the job remains in the BullMQ failed set (dead-letter) (per phase-03-videos/TD-01 , phase-03-videos/TD-08)

Dependency Map

```
SI-03.1 (no deps – infra: Compose, config namespaces, dependencies)
├─ SI-03.2 (Video entity + migration + VideosModule)
│   └─ SI-03.3 (upload initiation: draft + presigned multipart)
│       └─ SI-03.4 (BullMQ queue + worker: ffprobe/ffmpeg, status lifecycle)
│           └─ SI-03.5 (streaming + download: presigned GET, Range/206)
│               └─ SI-03.6 (complementary REST: get, list, update, delete)
```

Linearized implementation order: SI-03.1 → SI-03.2 → SI-03.3 → SI-03.4 → SI-03.5 → SI-03.6

The chain is strictly linear: infrastructure (SI-03.1) underpins the entity (SI-03.2); the upload handshake (SI-03.3) needs the persisted draft ; background processing (SI-03.4) is triggered on upload completion; streaming (SI-03.5) is gated on the ready status the worker produces; and the complementary CRUD (SI-03.6) reuses the storage/service layer built by the earlier steps.

Deliverables

- [] Docker Compose extended with redis (7-alpine, AOF), minio (+ minio-setup bucket bootstrap), and a nestjs-worker service (shared image, worker entrypoint, ffmpeg/ffprobe)
- [] Config namespaces storage , queue , upload via registerAs , validated by the Joi schema; .env.example updated
- [] Video entity + migration — internal uuid PK, unique nanoid public_id , video_status enum, storage/metadata columns (bigint file size, jsonb metadata)
- [] Presigned multipart upload (TD-03): POST /videos (draft + CreateMultipartUpload), per-part presigned URL, complete (CompleteMultipartUpload → processing) — 10GB bytes bypass the API
- [] StorageService over AWS SDK v3 against MinIO (forcePathStyle), covering multipart, presigned GET, and object delete
- [] BullMQ video-processing queue (TD-01) + isolated nestjs-worker (TD-04) consuming process-video
- [] Automatic processing: ffprobe duration/metadata + ffmpeg thumbnail via direct child_process (TD-05), thumbnail persisted to storage
- [] Status lifecycle draft → processing → ready | error (TD-08) with 3-attempt exponential-back-off retry and failed-set dead-letter
- [] Unique public video URL id via nanoid (TD-06)
- [] Streaming via presigned GET with native Range / 206 (TD-07) — direct client↔storage, gated on ready
- [] Download via presigned GET with Content-Disposition: attachment (TD-07)

- [] Complementary REST: retrieve (`ready -gated`), list by channel, owner-only update (title/description), owner-only delete with storage cleanup + multipart abort
- [] Standardized video-domain error catalog extending the inherited `DomainException` / `DomainExceptionFilter`
- [] All SI tests pass (`docker compose exec nestjs-api npm test -- --runInBand`)
- [] E2E tests pass (`docker compose exec nestjs-api npm run test:e2e`)
- [] Type/compilation check passes (`docker compose exec nestjs-api npx tsc --noEmit`)
- [] Project builds successfully (`docker compose exec nestjs-api npm run build`)